RAG 교안 04 · 5부~6부

임베딩 만들기와 ChromaDB 저장

청크의 글을 숫자 목록으로 바꾸고, 글·숫자·출처를 같은 ID로 ChromaDB에 저장합니다 (NB02 코드 1~11)

NB01 청크 나누기 노트북 · 앞 교안(03.Chunk_만들기)
NB02 임베딩·ChromaDB 저장 노트북 · 이 교안에서 실습
NB03 검색 노트북 · 다음 교안(05.RAG_검색)

수업 순서 Ollama GPU 설정 (2~6p) → 임베딩 개념 (7p) → NB02 코드 0~11 (8~24p) → 활동·다음 교안 연결 (25~28p)

---

qwen3-embedding:0.6b · Ollama · ChromaDB · skhynix_basic 컬렉션

Vulkan은 GPU를 쓰는 또 하나의 경로입니다

모델(qwen3-embedding:0.6b)은 그대로 두고, 임베딩을 계산하는 장치만 CPU에서 GPU로 바꿉니다. 아래 3단계를 순서대로 진행합니다.

1 설정

환경변수 OLLAMA_VULKAN=1, OLLAMA_LLM_LIBRARY=vulkan 저장

2 재시작

Ollama를 완전히 종료하고 다시 실행 (환경변수는 시작 시에만 읽음)

3 확인

임베딩 한 번 요청 후 ollama ps에서 PROCESSOR = 100% GPU 확인

Vulkan이란

GPU에 계산을 맡기는 공용 규격(API)입니다. Ollama는 기본적으로 NVIDIA는 CUDA, AMD는 ROCm 방식으로 GPU를 쓰는데, Vulkan은 제조사와 상관없이 GPU를 쓰는 대안 경로입니다.

왜 켜는지

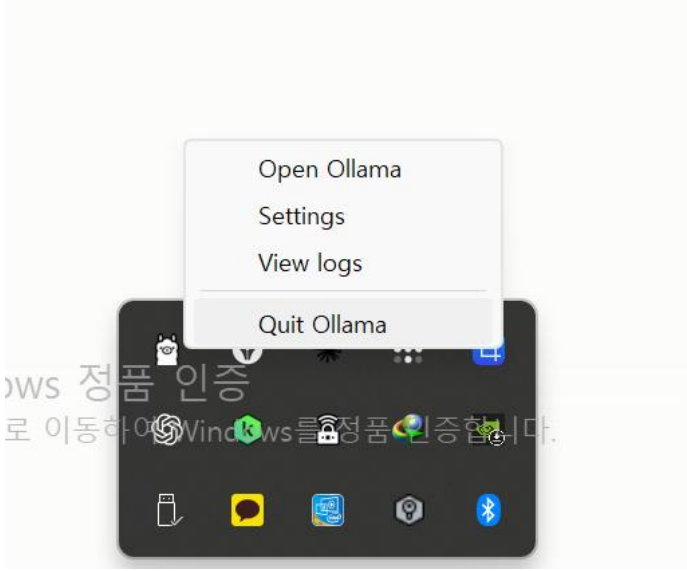
CUDA·ROCm 드라이버가 없거나 버전이 맞지 않는 PC에서도 GPU로 임베딩을 돌리기 위해서입니다. CPU만 쓰면 청크 800개 임베딩(본편 코드 9)이 훨씬 오래 걸립니다.

전제 Ollama 앱과 qwen3-embedding:0.6b는 설치되어 있어야 합니다. GPU가 없는 PC에서는 이 보충을 건너뛰고 본편으로 갑니다.

다음 → PowerShell에서 Vulkan 환경변수 두 개를 저장합니다

1단계 · 환경변수 두 개를 사용자 환경에 저장합니다

PowerShell에서 환경변수 OLLAMA_VULKAN=1과 OLLAMA_LLM_LIBRARY=vulkan을 사용자 환경에 저장합니다. 이 값은 Ollama 서버가 시작될 때 읽어 GPU 계산 경로를 Vulkan으로 바꿉니다.



PowerShell에 아래 **두 줄을 복사해서 실행**하면 사용자 환경변수에 영구 저장됩니다. 관리자 권한은 필요 없습니다.

```powershell
[Environment]::SetEnvironmentVariable('OLLAMA_VULKAN', '1', 'User')
[Environment]::SetEnvironmentVariable('OLLAMA_LLM_LIBRARY', 'vulkan', 'User')
```

저장됐는지 확인하려면:

```powershell
[Environment]::GetEnvironmentVariable('OLLAMA_VULKAN', 'User')
[Environment]::GetEnvironmentVariable('OLLAMA_LLM_LIBRARY', 'User')
```

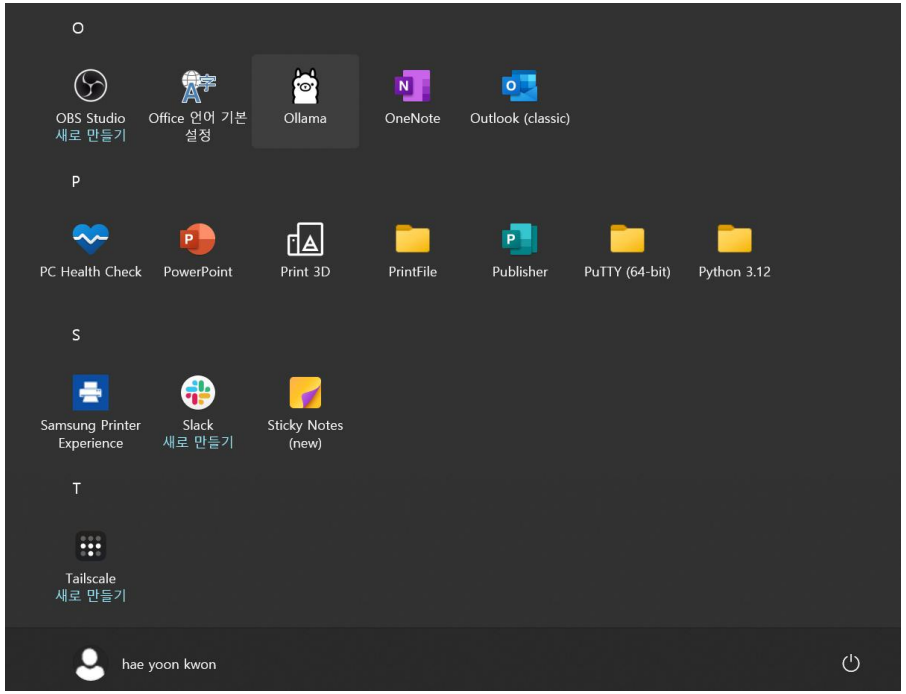
아래처럼 나오면 정상입니다.

```일반 텍스트
1
vulkan
```

저장 후 **실행 중인 Ollama를 완전히 종료하고, 시작 메뉴에서 Ollama를 다시 실행**하면 적용됩니다. 이후 PowerShell 창은 닫아도 됩니다.

2단계 · 환경변수는 Ollama를 재시작해야 적용됩니다

환경변수는 서버가 시작될 때만 읽으므로, 트레이의 Ollama를 Quit으로 완전히 종료한 뒤 시작 메뉴에서 다시 실행합니다.



주의 PowerShell 창을 닫는 것으로는 적용되지 않습니다. 재시작 후 로그(View logs)에 vulkan 백엔드가 표시되면 정상입니다.

다음 → 새 PowerShell 창에서 임베딩을 요청하는 스크립트를 실행합니다

3단계 · 임베딩을 한 번 요청하고 적재 장치를 확인합니다

새 PowerShell 창에서 아래 스크립트를 실행합니다. keep_alive로 모델을 10분 유지하고, embeddings[0].Count로 벡터 길이, ollama ps로 모델이 올라간 장치를 확인합니다.

powershell

```powershell
$body = @{
    model      = 'qwen3-embedding:0.6b'
    input      = 'GPU embedding test'
    keep_alive = '10m'
} | ConvertTo-Json

$result = Invoke-RestMethod `
    -Uri 'http://127.0.0.1:11434/api/embed' `
    -Method Post -ContentType 'application/json' -Body $body

$result.embeddings[0].Count
ollama ps
```

다음 → ollama ps 출력에서 모델의 GPU 적재를 확인합니다

ollama ps의 GPU 적재 상태

앞 스크립트의 마지막 줄 ollama ps 출력입니다. 임베딩 요청 직후 qwen3-embedding:0.6b가 100% GPU로 표시됩니다.

실행 결과

```
NAME                    ID              SIZE      PROCESSOR    CONTEXT    UNTIL
qwen3-embedding:0.6b    ac6da0dfba84    1.4 GB    100% GPU     4096       29 minutes from now
```

보충 3 스크립트 · ollama ps 실행 결과

100% GPU

PROCESSOR 열이 100% GPU이면 Vulkan 설정이 적용된 것입니다. 적용 전이라면 100% CPU 또는 CPU/GPU 혼합으로 표시됩니다. GPU 사용률이 항상 100%라는 뜻은 아닙니다.

1024

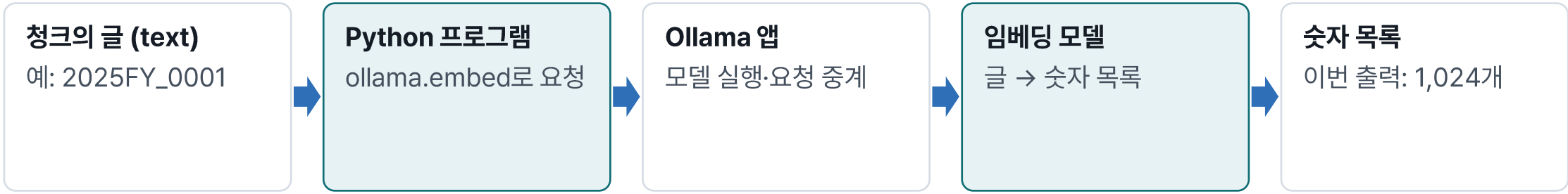
스크립트 첫 출력 1024는 벡터 한 개의 길이(차원 수)입니다. 본편 실습에서도 같은 값이 나옵니다.

유지 keep_alive 시간(스크립트 10m)이 지나면 모델이 내려갑니다. UNTIL 열의 시간이 지났으면 임베딩을 다시 요청한 뒤 확인합니다.

다음 → 노트북을 열기 전에 임베딩이 무엇인지 봅니다

임베딩 모델은 글을 비교할 숫자 목록으로 바꿉니다

도식



아닌 것

모델을 추가로 학습시키는 과정이 아닙니다. 답변 문장을 만드는 과정도 아닙니다.

숫자의 뜻

숫자 하나가 단어 하나의 뜻을 그대로 나타내지 않습니다. 목록 전체를 다른 목록과 비교하는 데 씁니다.

구분 Ollama 앱과 Python의 ollama 패키지는 다른 설치물입니다. 프로그램은 패키지를 통해 앱에 요청을 보냅니다.

다음 → 이 노트북에서 쓸 chromadb·ollama 패키지를 설치합니다

이 노트북이 쓸 두 패키지를 설치합니다

chromadb는 저장, ollama는 임베딩 요청에 씁니다. 앞에서 본 Ollama 앱과는 별개인 Python 패키지입니다.

실행 결과

```
%pip install -q chromadb ollama
```

[4] ✓ 1.4s

NB02 코드 0 · 실행 결과 (셀 번호 [4]는 실행 순서이며 교안의 코드 번호와 다릅니다)

-q 설치 메시지를 줄이는 옵션입니다. 없어도 결과는 같습니다.
1.4s 이미 설치된 환경의 시간입니다. 처음 설치하면 수십 초에서 수 분이 걸리고 출력이 길게 나올 수 있습니다.
확인 셀 왼쪽에 체크 표시가 나오면 설치가 끝난 것입니다. 오류 메시지가 빨간 글씨로 나오면 인터넷 연결을 확인한 뒤 다시 실행합니다.

주의 설치 후 커널을 재시작하라는 메시지가 나오면 재시작한 뒤 다음 셀부터 실행합니다.

다음 → 프로그램이 파일을 찾을 폴더를 정합니다

프로그램이 파일을 찾을 폴더를 정합니다

프로그램은 현재 작업 폴더 안의 data와 artifacts를 가리키는 이름을 만들고, 현재 작업 폴더의 전체 위치를 화면에 보여 줍니다. 파일 안의 글은 아직 읽지 않습니다.

실행 결과

[1]:
```
# 폴더와 파일의 위치를 다루는 Path 도구를 가져옵니다. import는 준비된 도구를 불러온다는 뜻입니다.
from pathlib import Path

# 현재 작업 폴더 안의 data를 가리킵니다. =는 오른쪽 값을 왼쪽 이름에 담으라는 뜻입니다.
data_folder = Path("data")
# 프로그램이 만든 청크·설정·테스트 결과를 둘 위치입니다. 이 줄만으로 폴더가 생기지는 않습니다.
output_folder = Path("artifacts")
# 현재 작업 폴더의 전체 경로를 반환합니다. 마지막 줄의 값은 Jupyter가 화면에 보여 줍니다.
Path.cwd()
```

[1]:
```
WindowsPath('I:/교육자료/한국폴리텍대학교_하이테크_2026_2학기/강의/prompt/work/rag_chunk_save_rewrite_20260921/run/rag_exam')
```

NB02 코드 1 · 실행 결과 (NB01 코드 1과 같은 코드)

결과 data_folder, output_folder라는 이름과 현재 폴더 표시. 이 셀은 파일 안의 글을 읽지 않습니다. **확인** 표시된 위치가 실습 폴더인지 봅니다.

주의 Path("data")는 현재 작업 폴더 안의 data라는 뜻입니다. 현재 위치가 다르면 같은 코드가 다른 폴더를 가리킵니다.

다음 → chunks.jsonl을 읽어 청크 목록을 다시 만듭니다

이 노트북은 chunks.jsonl을 읽어 청크 목록을 다시 만듭니다

이 노트북은 같은 작업 폴더의 artifacts/chunks.jsonl을 한 줄씩 읽고, 각 줄을 JSON에서 딕셔너리로 되돌려 chunks 목록에 넣습니다. 코드 속 01은 앞 노트북(NB01)을 가리킵니다.

실행 결과

[2]:
```python
# 파일 속 JSON 글을 딕셔너리로 되돌리는 도구입니다.
import json

# 파일에서 다시 읽은 청크들을 담을 목록입니다.
chunks = []
# 01이 저장한 chunks.jsonl을 읽기 모드로 엽니다.
with (output_folder / "chunks.jsonl").open(encoding="utf-8") as file:
    # 800줄을 한 줄씩 읽습니다. 한 줄은 청크 한 개입니다.
    for line in file:
        # 그 줄의 JSON 글을 딕셔너리로 바꿔 목록에 넣습니다.
        chunks.append(json.loads(line))
```

NB02 코드 2 · 실행 결과

확인할 점

입력 chunks.jsonl 800줄

결과 chunks 목록 800개 (다음 셀에서 len(chunks)로 확인)

주의 파일이 없으면 앞 노트북(NB01)이 먼저 저장했는지, 작업 폴더가 같은지 확인합니다.

연결

앞 노트북(NB01)의 마지막 결과 파일이 이 노트북의 첫 입력입니다. 두 노트북은 변수를 자동으로 공유하지 않습니다.

다음 → 임베딩 모델 이름을 코드에 넣습니다

프로그램이 사용할 임베딩 모델 이름을 정합니다

프로그램은 ollama 패키지를 불러오고, 요청에 쓸 모델 이름 qwen3-embedding:0.6b를 model에 넣습니다. 이 셀은 아직 요청을 보내지 않습니다.

실행 결과

[3]:
```python
# Python 코드에서 이 PC의 Ollama에 요청할 도구를 가져옵니다.
import ollama

# 문서와 질문을 같은 임베딩 모델로 처리하기 위해 이름을 정합니다.
model = "qwen3-embedding:0.6b"
```

NB02 코드 3 · 실행 결과

확인할 점

model "qwen3-embedding:0.6b"

0.6b 모델 크기 표시입니다. 파일 크기 0.6GB라는 뜻이 아닙니다.

같은 이름 다음 노트북(NB03)의 검색도 같은 모델을 씁니다. 모델 이름은 24p에서 파일로 저장합니다.

모델이 없으면 다른 모델로 임의 대체하지 않고 설치 목록(ollama list)을 확인합니다.

연결 2~6p GPU 설정에서 쓴 것과 같은 모델 이름이 여기서 코드에 들어갑니다.

다음 → 첫 청크의 글을 보내고 숫자 목록을 받는 코드를 봅니다

첫 청크의 글을 보내고 숫자 목록을 받습니다

Python 프로그램은 chunks[0]["text"]를 Ollama에 보냅니다. 임베딩 모델은 이 글에 대응하는 숫자 목록을 만들고, 프로그램은 첫 목록을 sample_vector에 담습니다.

실행 결과

[4]:
```
# input으로 글을 보내고 그 결과를 response에 받습니다. truncate=False는 긴 입력을 조용히 잘라 버리지 않게 합니다.
response = ollama.embed(model=model, input=chunks[0]["text"], truncate=False)
# 첫 번째 입력 글에 대응하는 숫자 목록만 꺼냅니다. 아직 DB에 저장하지는 않습니다.
sample_vector = response.embeddings[0]
```

NB02 코드 4 · 실행 결과

입력 첫 청크 2025FY_0001의 text **truncate=False** 입력이 너무 길면 조용히 잘라 처리하지 않고 오류로 알리게 하는 설정
response.embeddings[0] 첫 입력 글에 대응하는 숫자 목록 **아직 저장 전** 이 셀은 DB에 넣지 않습니다.

담당 프로그램이 보내고, Ollama가 실행한 임베딩 모델이 만들고, 프로그램이 받습니다. 세 담당을 구분해서 말합니다.

다음 → 받은 숫자 목록의 길이를 확인합니다

숫자 목록에 숫자가 몇 개 있는지 확인합니다

len(sample_vector)는 숫자의 합이 아니라 개수를 보여 줍니다. 이번 실행의 출력은 1024입니다.

실행 결과

```
[5]: # 숫자 목록의 길이를 화면에 보여 줍니다. 숫자를 합하거나 평균 내는 것이 아닙니다.
     len(sample_vector)

[5]: 1024
```

NB02 코드 5 · 실행 결과

1,024 이번 모델이 글 하나에 대해 만든 숫자의 개수
아닌 것 글자 수, 청크 수, 정답 수가 아닙니다.
새 실행 값이 다르면 모델 태그·실제 모델·요청 설정을 확인합니다.

구분 700은 본문을 나누는 글자 수 설정, 1,024는 임베딩 결과의 숫자 개수입니다. 서로 다른 것의 크기입니다.

다음 → 글 안의 기간과 별도 기간 필드가 어떻게 다르게 쓰이는지 봅니다

글 속의 기간과 별도 기간 필드는 쓰임이 다릅니다

프로그램은 청크의 글에도 기간과 절 제목을 붙이고, 같은 정보를 별도 이름표(metadata)에도 저장합니다. 두 경로의 용도가 다릅니다.

저장된 청크 2025Q3_0196 · text 필드 **(임베딩 입력)**

SK하이닉스 2025Q3
III. 재무에 관한 사항 > 2-2. 연결 포괄손익계산서
연결 포괄손익계산서	...	(단위 : 백만원)
	제 78 기 3분기 3개월	제 78 기 3분기 누적	...
매출액 (주4,21,28)	24,448,929	64,320,022	...

→ 임베딩 모델은 이 글 전체를 숫자로 바꿉니다. 기간 2025Q3와 절 제목도 글에 들어 있으므로 숫자에 반영됩니다.

저장된 청크 같은 청크 · metadata 필드 **(임베딩 제외)**

source = 000660_SK하이닉스_2025Q3_원문발췌_실습용_v1.0.md
period = 2025Q3
section = III. 재무에 관한 사항 > 2-2. 연결 포괄손익계산서

→ 다음 노트북(NB03)은 period 필드를 조회 조건(where)으로 쓰고, source·section을 결과 표시에 씁니다. 이 필드는 임베딩 모델에 보내지 않습니다.

id와 text는 metadata 안에 넣지 않습니다. 현재 metadata의 필드는 source, period, section 세 개입니다(NB02 코드 9). 같은 기간·절을 별도 필드에도 넣는 이유는 조건 검색과 결과 표시에 쓰기 위해서입니다. '내용'과 '이름표'라는 말은 이 두 경로를 구분하는 이름입니다.

주의 모델에 보내는 것은 texts 목록입니다. 이름표 필드는 보내지 않습니다.

다음 → 숫자를 저장할 ChromaDB 폴더와 컬렉션을 준비합니다

chroma_db 폴더의 전체 경로로 ChromaDB에 연결합니다

프로그램은 현재 작업 폴더 아래 chroma_db의 전체 경로를 resolve()로 구하고, PersistentClient로 그 위치의 DB에 연결합니다. 폴더가 없으면 새로 만듭니다.

실행 결과

[6]:
```
# 디스크에 원문과 임베딩을 저장하고 검색하는 ChromaDB 도구입니다.
import chromadb

# chroma_db라는 상대경로의 실제 전체 위치를 실행 중 계산합니다.
db_folder = Path("chroma_db").resolve()
# 그 위치의 DB를 사용하기 위한 연결 객체를 만듭니다. Persistent는 디스크에 남긴다는 뜻입니다.
client = chromadb.PersistentClient(path=str(db_folder))
```

NB02 코드 6 · 실행 결과

Persistent 디스크에 남긴다는 뜻입니다. 커널을 끝내도 저장 내용이 남습니다. **경로** 실행 복사본에서는 그 복사본 아래 chroma_db가 됩니다. 작업 폴더가 다르면 다른 DB에 연결됩니다. **첫 실행** 폴더가 없으면 새로 만듭니다.

이유 메모리의 변수는 커널을 끝내면 사라집니다. 다음 노트북(NB03)에서 쓰려면 디스크 DB가 필요합니다.

다음 → 자료를 담을 컬렉션을 준비합니다

skhynix_basic 컬렉션을 열거나 새로 만듭니다

컬렉션은 관련 항목을 묶어 두는 보관 단위입니다. 프로그램은 skhynix_basic 컬렉션을 같은 이름이 있으면 열고, 없으면 새로 만듭니다.

실행 결과

[7]:
```python
# 02에서 저장하고 03·05에서 열 보관함의 이름입니다.
collection_name = "skhynix_basic"
# 같은 이름이 있으면 열고, 없을 때만 새로 만듭니다.
collection = client.get_or_create_collection(
    # 보관함 이름을 전달합니다.
    name=collection_name,
    # ChromaDB가 글을 자동 임베딩하지 않게 합니다. 뒤에서 Ollama 결과를 직접 전달합니다.
    embedding_function=None,
    # 새 컬렉션에서는 cosine 거리를 사용하도록 설정합니다. hnsw는 검색용 내부 색인 설정 이름입니다.
    configuration={"hnsw": {"space": "cosine"}}
)
```

NB02 코드 7 · 실행 결과

embedding_function=None 자동 임베딩 대신 프로그램이 Ollama에서 받은 숫자를 직접 전달한다는 뜻입니다('임베딩을 쓰지 않음'이 아님).
cosine 숫자 목록을 비교하는 거리 설정이며 새 컬렉션을 만들 때 적용됩니다.

구분 '이름표 붙은 보관함' 비유는 저장 구조를 설명합니다. 보관함이 자료의 진위를 판단하지는 않습니다. 실제 이름은 ChromaDB의 컬렉션입니다.

다음 → 저장할 네 목록이 같은 ID로 어떻게 연결되는지 봅니다

같은 ID로 글·숫자·출처를 연결합니다

프로그램은 ID, 글, 메타데이터, 임베딩 숫자를 각각 목록으로 만듭니다. 네 목록의 같은 위치는 같은 청크를 가리킵니다. ChromaDB는 이 정보를 ID에 연결해 저장합니다.

저장 결과 skhynix_basic 컬렉션에 저장된 첫 청크 2025FY_0001의 값

코드 9의 목록	upsert 인자	첫 청크의 실제 저장값
ids	ids	2025FY_0001
texts	documents	SK하이닉스 2025FY\n공시 개요\n\n# 사업보고서 (28자)
metadata	metadatas	{source: 000660_SK하이닉스_2025FY_원문발췌_실습용_v1.0.md, period: 2025FY, section: 공시 개요}
response.embeddings	embeddings	[0.0801, -0.0010, -0.0071, ...] 숫자 1,024개

다음 검색에서 사용자는 숫자뿐 아니라 연결된 글과 출처를 볼 수 있습니다. 네 목록의 순서가 어긋나면 다른 글의 출처가 연결됩니다.

표시 DB에 저장된 첫 청크 2025FY_0001의 값입니다. 숫자 목록은 앞 세 개만 표시했습니다.

다음 → 다시 실행할 때 기존 항목을 어떻게 지우는지 봅니다

다시 실행하면 이 컬렉션의 기존 항목을 지웁니다

프로그램은 skhynix_basic 컬렉션의 기존 ID 목록을 읽습니다. 기존 항목이 있으면 그 ID에 연결된 자료를 지웁니다. 뒤 단계에서 입력 청크 전체를 다시 임베딩해 저장합니다.

실행 결과

[8]:
```
# 본문이나 임베딩은 받지 않고 현재 보관함의 ID 목록을 읽습니다.
old_ids = collection.get(include=[])["ids"]
# 목록이 비어 있지 않은 경우에만 삭제합니다.
if old_ids:
    # 이 보관함에서 해당 ID의 자료를 삭제합니다. 원문 MD 파일을 삭제하는 코드가 아닙니다.
    collection.delete(ids=old_ids)
```

NB02 코드 8 · 실행 결과

지우는 범위 이 컬렉션의 항목만. 원문 MD, chunks.jsonl, 다른 컬렉션은 지우지 않습니다. **처음 실행** 새 DB에는 지울 항목이 없어 if 조건이 실행되지 않습니다. **다시 실행하면 기존 800개가 지워지고 다시 저장됩니다.** 이전 임베딩을 재사용하지 않습니다.

주의 중간에 실패하면 DB가 비어 있거나 일부만 저장될 수 있습니다. 코드 10의 개수 확인까지 마쳐야 저장이 끝난 것입니다.

다음 → 청크를 16개씩 묶어 요청하는 코드를 봅니다

청크를 최대 16개씩 묶습니다

프로그램은 시작 번호 0, 16, 32, ...로 청크 목록을 최대 16개씩 묶습니다. 묶음마다 ids, texts, metadata 빈 목록을 준비합니다.

실행 결과

```
[9]: # 시작 번호는 0, 16, 32, ...입니다. 한 번에 16개씩 임베딩 요청합니다.
for start in range(0, len(chunks), 16):
    # 예: 첫 반복은 0번부터 15번까지, 다음 반복은 16번부터 31번까지입니다.
    batch = chunks[start:start + 16]
    # 이번 묶음의 ID·본문·이름표를 각각 담을 빈 목록을 만듭니다.
    ids = []
    texts = []
    metadata = []
```

NB02 코드 9 · 1~8행 · 실행 결과

확인할 점

16 한 번에 요청할 청크 수. 글자 수·토큰 수가 아닙니다.

800개 16개씩 50묶음

마지막 묶음 남은 청크 수에 따라 16개보다 적을 수 있습니다.

최적값 아님 환경에 따라 다른 값을 쓸 수 있습니다.

이유 800개를 한 번에 보내면 요청이 커집니다. 묶음으로 나누면 한 요청의 크기를 줄일 수 있습니다.

다음 → 묶음 안의 청크를 세 목록에 같은 순서로 담는 부분을 봅니다

묶음 안의 청크를 같은 순서로 세 목록에 담습니다

프로그램은 묶음 안의 청크를 순서대로 읽어 ids에 id, texts에 text, metadata에 source·period·section을 넣습니다.

실행 결과

```
    # 묶음 안의 청크를 순서대로 읽어 세 목록의 순서를 맞춥니다.
    for chunk in batch:
        # 청크 ID를 넣습니다. 예: 2025FY_0001.
        ids.append(chunk["id"])
        # 임베딩 모델에 보낼 글을 넣습니다.
        texts.append(chunk["text"])
        # 검색 결과에서 출처와 기간을 확인할 이름표를 넣습니다.
        metadata.append({
            # 출처 파일과 보고기간을 그대로 보관합니다.
            "source": chunk["source"], "period": chunk["period"],
            # 원문에서 어느 장·절인지도 함께 보관합니다.
            "section": chunk["section"]
        })
```

NB02 코드 9 · 9~21행 · 실행 결과

확인할 점

같은 순서 세 목록의 n번째는 같은 청크입니다.

metadata 필드 source, period, section (id·text는 넣지 않음)

이름 코드의 변수는 metadata, API 인자는 metadatas입니다.

이 화면 코드 9의 9~21행

확인 순서가 어긋나면 다른 글의 출처가 연결됩니다. 프로그램은 같은 청크의 정보를 같은 순서로 모아야 합니다.

다음 → 묶음을 임베딩하고 upsert로 저장하는 부분을 봅니다

묶음을 임베딩하고 upsert로 함께 저장합니다

임베딩 모델은 texts의 각 글에 대응하는 숫자 목록을 만듭니다. 프로그램은 그 결과와 ids, documents, metadatas를 upsert에 함께 전달합니다.

실행 결과

```
    # 이번 texts의 각 글을 같은 순서의 숫자 목록으로 바꿉니다.
    response = ollama.embed(model=model, input=texts, truncate=False)
    # ID·본문·이름표·임베딩을 한꺼번에 DB에 넣습니다.
    collection.upsert(
        # 네 목록의 같은 번호끼리 한 청크로 연결됩니다.
        ids=ids, documents=texts, metadatas=metadata,
        # Ollama가 돌려준 숫자 목록을 직접 전달합니다.
        embeddings=response.embeddings
    )
```

NB02 코드 9 · 22~30행 · 실행 결과

확인할 점

upsert 같은 ID가 있으면 갱신, 없으면 추가

이번 노트북 앞에서 기존 항목을 지웠으므로 전체 재구축입니다.

담당 모델은 숫자를 만들고, 저장은 프로그램이 ChromaDB에 요청합니다.

반복 50묶음이 끝날 때까지 이 셀이 이어집니다(이번 실행 약 167초).

구분 upsert라는 이름과 달리 이 노트북은 비운 뒤 전체를 저장합니다.

다음 → 입력 개수와 DB 개수를 비교합니다

입력 청크 수와 DB 항목 수를 비교합니다

프로그램은 파일에서 읽은 청크 수와 DB에 저장된 항목 수를 보여 줍니다. 학생은 두 수가 같은지 확인합니다. 이번 실행의 출력은 입력 800, DB 800입니다.

실행 결과

```
[10]: # 왼쪽 값은 Python 목록의 길이, 오른쪽 값은 DB 안 항목 수입니다.
      {"입력 청크": len(chunks), "DB 청크": collection.count()}

[10]: {'입력 청크': 800, 'DB 청크': 800}
```

NB02 코드 10 · 실행 결과

확인할 점

개수가 같으면 저장 수량은 확인된 것입니다.

개수가 다르면 저장 완료 여부와 DB 경로를 확인합니다.

한계 개수가 같아도 내용 연결과 검색 품질은 따로 확인해야 합니다.

확인 이 화면은 항목 수의 확인입니다. 개수 확인만으로 검색이 정확하다고 볼 수는 없습니다.

다음 → 다음 검색이 읽을 설정 파일을 저장합니다

질문용 지시문(query_prefix)을 정합니다

프로그램은 다음 검색에서 질문 앞에 붙일 지시문을 query_prefix에 넣습니다. 이 지시문은 검색에 쓸 숫자를 만들 때 질문 앞에 붙는 글이며, 답변 문장을 생성하는 요청이 아닙니다.

실행 결과

```
[11]: # 검색 목적을 알려 주는 공통 글입니다. Query: 뒤에는 실제 질문이 붙습니다.
query_prefix = "Instruct: Given a question, retrieve passages from Korean financial disclosures that answer the question.\nQuery: "
```

NB02 코드 11 · 1~2행 · 실행 결과

지시문 전체 Instruct: Given a question, retrieve passages from Korean financial disclosures that answer the question. / Query:

뜻 '질문을 주면 한국어 금융 공시에서 그 질문에 답하는 글을 찾으라'는 안내입니다. 실제 질문은 Query: 뒤에 붙습니다.

구분 이 지시문은 다음 노트북(NB03)이 질문을 임베딩할 때 씁니다. 문서 청크를 임베딩할 때는 붙이지 않았습니다.

다음 → 설정 네 가지를 index.json에 저장하는 부분을 봅니다

모델·컬렉션·지시문·청크 수를 index.json에 저장합니다

프로그램은 모델 이름, 컬렉션 이름, 질문용 지시문, 청크 수를 settings 묶음으로 만들어 artifacts/index.json에 저장합니다. 다음 검색 프로그램은 이 파일을 읽습니다.

실행 결과

```
# 다른 노트북이 같은 설정을 읽도록 필요한 값만 한 묶음으로 만듭니다.
settings = {
    # 사용한 모델과 저장한 보관함의 이름입니다.
    "model": model, "collection": collection_name,
    # 질문용 공통 지시문과 기대하는 저장 개수입니다.
    "query_prefix": query_prefix, "chunk_count": len(chunks)
}
# 설정 결과 파일을 엽니다. 다음 노트북은 이 파일을 읽습니다.
with (output_folder / "index.json").open("w", encoding="utf-8") as file:
    # 한글을 유지하고 들여쓰기 두 칸으로 사람이 읽기 편하게 저장합니다.
    json.dump(settings, file, ensure_ascii=False, indent=2)
```

NB02 코드 11 · 3~13행 · 실행 결과

해당 실행 결과 · index.json의 네 필드

model qwen3-embedding:0.6b

collection skhynix_basic

query_prefix Instruct: … Query: (앞 화면)

chunk_count 800

아닌 것 임베딩 숫자가 들어 있는 파일이나 검색 색인 자체가 아닙니다. 설정 파일입니다.

이유 검색할 때 저장과 다른 모델이나 컬렉션을 쓰면 올바르게 비교하기 어렵습니다. 설정도 함께 남깁니다.

다음 → 숫자와 처리 과정을 구분하는 활동으로 정리합니다

활동. 설정값과 실제 결과를 구분해 봅니다

이 활동에서는 학생이 코드의 설정값과 실제 결과를 구분합니다. 각 질문에 답한 뒤 다음 화면의 해설과 비교합니다.

활동

1. 700자, 200자, 1,024개, 16개, 800개는 각각 무엇의 크기입니까?
2. 최종 청크가 900자라면 분할 코드가 반드시 잘못된 것입니까?
3. 앞 두 문단 중 하나가 260자, 다른 하나가 100자라면 어떤 앞 설명을 선택합니까?
4. 한 묶음의 texts 순서와 metadata 순서가 다르면 어떤 문제가 생깁니까?
5. 이 노트북을 다시 실행하면 변경된 청크만 갱신합니까?
6. JSONL과 DB에 800개가 있다는 사실만으로 검색이 정확하다고 판단할 수 있습니까?
7. metadata의 period와 청크 글 속의 '2025Q3'는 각각 어디에 쓰입니까?

활동에 쓸 자료

NB01(앞 교안) 코드 8 · 250자 미만 조건

NB01(앞 교안) 코드 10 · chunk_size 700, chunk_overlap 200

NB01(앞 교안) 코드 11 · text 결합 순서

NB02 코드 5 출력 · 1024

NB02 코드 8 · 기존 ID 삭제

NB02 코드 9 · 16개 묶음, metadata 세 필드

NB02 코드 10 출력 · 입력 800, DB 800

NB03(다음 교안) · where={"period": period}

진행 답과 이유는 다음 화면에 있습니다.

다음 → 답과 이유를 확인합니다

해설. 무엇의 크기이며 누가 만드는지 함께 말합니다

해설

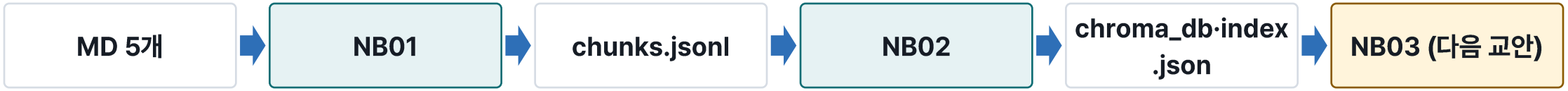
1. 700은 본문을 나누는 글자 수 설정, 200은 겹침 목표 설정, 1,024는 임베딩 결과의 숫자 개수, 16은 한 번에 요청하는 청크 수, 800은 현재 전체 청크 수입니다.
2. 아닙니다. 프로그램이 기간·제목·앞 설명·머리글을 붙이므로 최종 길이는 더 길 수 있습니다. 학생은 본문 분할과 최종 결합을 구분해 확인합니다.
3. 프로그램은 250자 미만 조건에 맞는 100자 문단만 선택합니다. 더 멀리서 다른 문단을 찾아 채우는 코드는 없습니다.
4. 다른 글의 출처 정보가 연결될 수 있습니다. 프로그램은 같은 청크의 정보를 같은 순서로 모아야 합니다.
5. 아닙니다. 현재 코드는 대상 컬렉션을 비운 뒤 전체를 다시 임베딩해 저장합니다.
6. 아닙니다. 개수는 저장 수량의 확인입니다. 학생은 내용 연결과 이후 검색 결과도 확인해야 합니다.
7. metadata의 period는 다음 검색에서 조회 범위 조건에 쓰입니다. 글 속의 2025Q3는 글의 문맥이자 임베딩 모델이 받는 입력의 일부입니다.

마무리 숫자만이 아니라 '무엇의 크기이며 누가 만드는지'를 말할 수 있으면 이 교안의 목표에 도달한 것입니다.

다음 → 준비 결과가 다음 검색과 어떻게 이어지는지 봅니다

준비 결과와 다음 검색 연결

앞 노트북(NB01)은 chunks.jsonl을 만듭니다. 이 노트북(NB02)은 ChromaDB에 글과 숫자, 출처를 저장하고 index.json을 만듭니다. 다음 노트북(NB03)은 이 저장 자료를 읽어 질문에 가까운 후보를 찾습니다.



학습 순서	실습 노트북	이어지는 내용
1	—	필요성·역할·전체 흐름·실습 범위
2	—	Ollama 앱과 qwen3-embedding:0.6b 준비
3	NB01 (앞 교안)	MD 문서를 청크로 나누어 chunks.jsonl 저장
4 (이 교안)	NB02	청크를 임베딩해 ChromaDB에 저장, index.json 작성
5	NB03 (다음 교안)	질문과 기간(예: 2025Q3) 입력, 후보 조회

다음 노트북(NB03)에서 질문과 조회 기간을 입력합니다. 다음 노트북은 index.json의 모델·컬렉션·지시문과 chroma_db를 읽습니다. 이 노트북에는 질문 검색·평가·답변 생성 코드가 없습니다.

> **안내** 이 교안은 4단계(NB02)입니다. 다음 교안(5단계, NB03)에서 저장한 결과를 검색에 사용합니다.

다음 → 실행 중 문제가 생기면 먼저 확인할 위치를 봅니다

실행 중 문제가 생기면 먼저 확인할 위치

문제가 생기면 확인할 파일·셀·값과 다음 행동입니다.

현상	먼저 확인할 위치	확인 방향
MD 파일 목록이 비어 있음	NB01(앞 교안) 코드 1의 현재 폴더, data 폴더	프로그램이 어느 폴더에서 파일을 찾는지 확인합니다.
chunks.jsonl을 찾지 못함	NB01(앞 교안) 코드 14 완료 여부, 작업 폴더	앞 노트북(NB01)이 저장한 파일이 이 노트북의 입력입니다.
chunks 등 이름 오류(NameError)	앞 코드 셀 실행 여부	노트북 파일 사이에 변수는 자동 공유되지 않습니다.
패키지 import 오류	커널: Python (stock_agent_env01)	다른 환경에 설치했는지 확인합니다.
Ollama 연결 실패	Ollama 앱 실행 여부, 요청 대상	DB 재구축부터 반복하지 않습니다.
모델을 찾지 못함	정확한 모델 태그와 ollama list	다른 모델로 임의 대체하지 않습니다.
저장 중 오류	실패한 셀, 실행 복사본 DB, 오류 내용	기존 자료 삭제 뒤 일부만 저장될 수 있습니다.
입력과 DB 개수가 다름	코드 9 완료 여부, DB 경로	개수 불일치를 숨기지 않고 확인합니다.
다음 검색이 다른 자료를 사용함	index.json, DB 경로, 컬렉션	설정 파일과 DB를 함께 이어서 사용합니다.